

Lezione — Modulo S4: Binary Exploits (Stack Overflow, Shellcode, ret2libc, ROP)

Corso: Lab Sicurezza Informatica T **Materiale:** `Binary_exploits_13_marzo.pdf` (teoria, Prandini) + `LAB_bruteforcing_e_buffer_overflows_18_marzo.pdf` (lab, Melis/Prandini) **Prerequisiti:** S1 (principi offensive, enumerazione) , S2 (autenticazione) . **Non serve** aver fatto corsi di Architetture o Sistemi Operativi: tutto ciò che serve sapere su memoria, stack e registri è costruito da zero nella *Parte 0* qui sotto. **Nota esame:** questo è uno dei **5 tipi di esercizio della prova pratica (60%)** — “Binary exploitation”. Ma conta anche per il **quiz teorico (40%)**, dove c’è **penalità per risposta sbagliata**: se un’affermazione su canary/NX/ASLR non ti è chiarissima, all’esame **non rispondere**.

Come leggere questa lezione

La lezione si regge su pochissimi oggetti concreti, e tutto il resto gira intorno a loro:

- **un pezzo di memoria** (lo *stack*) e **tre registri** del processore (ESP, EBP, EIP);
- **due funzioni C scritte male** (`gets` e `strcpy`) che copiano dati senza controllarne la lunghezza;
- **una stringa d’attacco** costruita “a mano” con `perl/python`;
- **un debugger** (`gdb`) per guardare dentro la memoria;
- **quattro binari** del lab (`write_var`, `secret_function`, `shellcode`, `returnlib`) che sono quattro gradini di difficoltà crescente;
- **tre flag di gcc** e **un file di sistema** (`/proc/sys/kernel/randomize_va_space`) che accendono e spengono le difese.

L’idea di fondo è una sola, e conviene fissarla subito: *se un programma copia in memoria un input più lungo del previsto senza controllare, quell’input “sborda” e va a sovrascrivere altri dati vicini — e se tra quei dati vicini c’è l’indirizzo a cui il programma tornerà, l’attaccante decide cosa il programma eseguirà dopo*. Tutto il modulo è capire **perché** quell’indirizzo è lì, **come** lo si sovrascrive e **come** ci si difende.

Questa lezione spiega il *perché*. Il *come*, *passo per passo* (lanciare `gdb`, trovare l’offset esatto, comporre il payload byte per byte) è la **guida-lab**, che farai con `/lab S4`. Qui non troverai la sequenza eseguibile: troverai il significato di ogni pezzo.

Parte 0 — Le fondamenta che il PDF dà per scontate

Il PDF di teoria parte dal presupposto che tu sappia già cos’è lo *stack*, cosa sono i registri e come una funzione viene chiamata in linguaggio macchina. Sono concetti dei corsi di Architetture e Sistemi Operativi. Li costruiamo qui da zero, perché **senza di essi il resto del modulo è incomprensibile**: gli attacchi binari *sono* manipolazione dello *stack*.

0.1 — Un programma in esecuzione: il processo e la sua memoria

Quando lanci un eseguibile, il sistema operativo ne crea un **processo**: una copia “viva” del programma con uno spazio di memoria tutto suo. Quel pezzo di RAM assegnato al processo non è un

blocco indistinto, ma è diviso in **segmenti**, ciascuno con un ruolo. Su Linux i principali sono:

- **.text** — il **codice** del programma, cioè le istruzioni in linguaggio macchina. È in sola lettura (non si dovrebbe modificare il proprio stesso codice).
- **.data** — le **variabili globali già inizializzate** (es. `char *bash = "/bin/bash";`).
- **.bss** — le **variabili globali non inizializzate**.
- **heap** — la memoria che il programma chiede *dinamicamente* mentre gira (la `malloc` del C). Cresce **verso indirizzi più alti**.
- **stack** — la memoria delle **funzioni**: le variabili locali e le informazioni per tornare indietro dopo una chiamata. Cresce **verso indirizzi più bassi** (su questo torniamo, è cruciale).

Un dettaglio importante: gli indirizzi che il programma “vede” sono **indirizzi virtuali**. Sono una finzione comoda che l’OS (con l’aiuto dell’hardware) traduce negli indirizzi fisici reali della RAM. Per l’attaccante conta solo lo spazio virtuale, perché è quello su cui il programma ragiona.

In piccolo (il meccanismo): sulla VM puoi vedere questa mappa con `pmap <PID>` (o `man pmap`). Ogni riga è un segmento con il suo range di indirizzi.

In grande (la visione): questa suddivisione è *la mappa del campo di battaglia*. Ogni attacco di questo modulo consiste nello scrivere in un segmento (lo stack) qualcosa che faccia saltare l’esecuzione in un altro punto (in **.text**, o in una libreria). Sapere **dov’è** ogni cosa è metà del lavoro — e infatti la principale difesa moderna (ASLR) consiste proprio nel **nascondere dove sono i segmenti**.

0.2 — Cos’è lo stack (da zero)

“Stack” significa **pila**, ed è esattamente la metafora giusta: una pila di piatti. Puoi fare due sole operazioni:

- **PUSH**: metti un nuovo elemento *in cima*;
- **POP**: togli l’elemento che sta *in cima*.

L’ultimo che entra è il primo che esce (LIFO, *Last In First Out*). Il processore usa questa pila per gestire le **chiamate a funzione**: ogni volta che una funzione viene chiamata, viene “impilato” un blocco di informazioni; quando la funzione finisce, quel blocco viene “spilato” e si torna a chi aveva chiamato.

Due fatti che a prima vista sembrano contraddittori, ma che sono **il cuore di tutta la vulnerabilità**:

1. **Lo stack cresce verso il basso.** Cioè ogni nuovo PUSH usa un indirizzo *più piccolo* del precedente. I dati “più vecchi” stanno in alto (indirizzi grandi), i “più nuovi” in basso (indirizzi piccoli).
2. **Un buffer, invece, si riempie verso l’alto.** Se dentro lo stack c’è un array `char buf[10]`, e lo riempi di caratteri, `buf[0]` sta in basso e `buf[9]` sta più in alto — cioè verso gli indirizzi *crescenti*, nella direzione dei dati più vecchi.

Tieni insieme questi due fatti: **riempiendo un buffer oltre la sua dimensione, l’eccesso non va “fuori” a caso, ma sale verso i dati impilati prima — esattamente dove sta l’informazione che dice alla funzione dove tornare.** È questa la collisione che si sfrutta.

0.3 — I registri del processore (architettura IA32)

I **registri** sono piccole celle di memoria velocissime *dentro* il processore (non nella RAM): il processore ci tiene i valori su cui sta lavorando in quell'istante. Il corso usa l'architettura **IA32** (Intel a 32 bit, cioè registri da 4 byte) perché è “non realistica ma comprensibile” (le idee valgono identiche sul 64 bit). I registri da conoscere:

- **EAX, EBX, ECX, EDX** — registri “general purpose”, per calcoli vari. Per convenzione, **EAX** è anche dove una funzione lascia il proprio **valore di ritorno**.
- **ESI, EDI** — “source” e “destination”, usati per copiare dati in memoria.
- **EIP** (*Instruction Pointer*) — **il registro più importante per noi**: contiene l'indirizzo della **prossima istruzione da eseguire**. Chi controlla EIP controlla il programma. *Tutto l'attacco serve a mettere in EIP un indirizzo scelto da noi*.
- **ESP** (*Stack Pointer*) — punta alla **cima dello stack** (l'ultima cella occupata). **PUSH** lo decrementa di 4 e scrive; **POP** legge e lo incrementa di 4.
- **EBP** (*Base Pointer*) — punta alla **base dello stack frame della funzione corrente**: è il punto di riferimento fisso rispetto a cui la funzione trova le proprie variabili locali e i propri parametri (es. `[EBP-4]` è una variabile locale, `[EBP+8]` è un parametro).

0.4 — Lo stack frame e la convenzione di chiamata

Quando una funzione chiama un'altra, sullo stack viene costruito uno **stack frame** (o *record di attivazione*): un blocco ordinato di dati. La regola con cui questo blocco è costruito si chiama **convenzione di chiamata**; quella di default in C su IA32 è `__cdecl`, e funziona così:

1. Il **chiamante** mette sullo stack i **parametri** della funzione, in ordine inverso (con dei **PUSH**).
2. Il chiamante esegue l'istruzione **CALL**, che fa due cose: impila l'**indirizzo di ritorno** (l'indirizzo dell'istruzione a cui tornare quando la funzione finirà) e salta dentro la funzione.
3. Il **chiamato**, appena entra, salva il vecchio EBP sullo stack (**PUSH EBP**) e imposta EBP sul valore attuale di ESP: così crea il proprio sistema di riferimento. Poi riserva spazio per le **variabili locali** (**SUB ESP, ...**).
4. Quando finisce, ripristina EBP e ESP, e infine esegue **RET**: questa istruzione fa il **POP dell'indirizzo di ritorno dallo stack dentro EIP** — e quindi l'esecuzione riprende da dove il chiamante l'aveva lasciata.

Fermati su questo punto, perché è **la chiave di volta dell'intero modulo**:

RET prende l'indirizzo di ritorno dallo stack e lo carica in EIP. L'indirizzo di ritorno è un *dato sullo stack*, vicino alle variabili locali e ai buffer. Se riusciamo a **sovrascrivere quel dato**, allora al **RET** il processore salterà **dove diciamo noi**, non dove il programma voleva tornare. Abbiamo dirottato l'esecuzione.

Ecco quindi la disposizione tipica di uno stack frame, dall'indirizzo alto (in cima) verso il basso:

```
indirizzi ALTI
+-----+
| Parametri funzione      |
+-----+
| Indirizzo di ritorno    | <-- se sovrascrivo QUESTO, controllo dove va il RET
+-----+
| EBP salvato             |
+-----+
```

```
| buffer / var. locali | <-- l'input "scorretto" finisce qui e sborda VERSO L'ALTO
+-----+
indirizzi BASSI (<-- qui punta ESP)
```

0.5 — Little-endian (perché gli indirizzi si scrivono “al contrario”)

Un ultimo tassello che servirà subito. Quando un numero da 4 byte (es. un indirizzo 0x0804D432) viene memorizzato in RAM, l'ordine dei byte dipende dall'architettura:

- **big-endian** (processori Motorola): si parte dal byte **più significativo** → 08 04 D4 32.
- **little-endian** (processori Intel, quindi il nostro caso): si parte dal byte **meno significativo** → 32 D4 04 08.

Conseguenza pratica: quando nella stringa d'attacco devi inserire l'indirizzo 0x0804D432, lo scrivi **byte invertiti**: \x32\xD4\x04\x08. Non è un vezzo, è come la CPU Intel rilegge quei 4 byte. Se li metti nell'ordine “naturale”, salti all'indirizzo sbagliato.

Con questo, le fondamenta sono complete. Da qui in poi ogni concetto si appoggia su Parte 0.

La visione d'insieme / threat model

Gli attacchi di questo modulo si chiamano **attacchi applicativi** o **binary exploit**: sfruttano vulnerabilità del *software in esecuzione locale* (e del modo in cui l'OS e l'architettura gestiscono la memoria), **non** falle di rete o di protocolli. Lo scopo generale di un exploit è **far eseguire a un processo operazioni per cui non era stato pensato**, con tre obiettivi possibili (non esclusivi):

- **fermarlo** → *Denial of Service* (DoS): il processo va in crash;
- **dirottarne il flusso di esecuzione** → eseguire codice scelto dall'attaccante;
- **ottenere i privilegi di altri utenti** → se il programma vulnerabile gira come root (o ha il bit SUID), prenderne i privilegi.

Perché esistono ancora queste falle? Perché la maggior parte del software di base è scritto in C/C++, linguaggi “vicini all'hardware” che **non controllano automaticamente** la lunghezza dei dati: il controllo è lasciato al programmatore, che spesso lo dimentica. È un problema così serio che — lo cita il PDF — nel febbraio 2024 la Casa Bianca ha esortato gli sviluppatori ad abbandonare C e C++ per linguaggi che inseriscono controlli automatici (come Rust o ADA).

Prospettiva dell'attaccante: cercare un input “non controllato e non validato” con cui scrivere su indirizzi di memoria arbitrari, e da lì arrivare a controllare EIP.

Prospettiva del difensore: il modulo non è solo offensivo. Buona metà del materiale sono le **contromisure** (canary, NX, ASLR, PIE/RelRO, CFI): capire *come* ogni difesa rende più difficile l'attacco — e *come* gli attaccanti la aggirano — è esattamente ciò che il quiz teorico chiede.

Lo stack overflow: il meccanismo centrale

Tutto nasce da qui. Le **precondizioni** sono due, semplici:

1. esiste un **buffer locale a una funzione** (quindi sta sullo stack), di dimensione fissa;
2. quel buffer si può **riempire con input esterni** (tastiera, file, rete).

Il **gesto dell'attacco**: l'attaccante immette un input più lungo del buffer. In assenza di controlli, i byte in eccesso “sbordano” (è l'*overflow*) e — risalendo verso gli indirizzi alti, come visto in 0.2 — vanno a sovrascrivere prima l'EBP salvato, poi l'**indirizzo di ritorno**. Al RET, il flusso non torna al chiamante legittimo ma all'indirizzo messo dall'attaccante.

I due “rubinetti aperti” tipici, da riconoscere a colpo d'occhio nel codice:

- **gets(buffer)** — legge una riga da standard input **senza alcun limite** di lunghezza. È intrinsecamente insicura.
- **strcpy(dest, src)** — copia **src** in **dest** finché non trova il terminatore, **senza controllare** che **dest** sia grande abbastanza.

In piccolo (il meccanismo): considera `void function(){ char buffer[10]; gets(buffer); }`. Se immetti `"AAAAAAAAAABBBBCCCC"`, le 10 A riempiono `buffer[10]`, le 4 B coprono l'EBP salvato, e le 4 C finiscono **esattamente sopra l'indirizzo di ritorno**. Quei 4 byte sono ciò che il RET caricherà in EIP.

In grande (la visione): la stringa d'attacco ha quindi una struttura precisa a tre zone: **[padding] + [riempimento dell'EBP] + [nuovo indirizzo di ritorno]**. Il padding serve solo a “consumare” lo spazio fino all'indirizzo di ritorno; ciò che conta è l'ultima zona. Trovare *quanti* byte di padding servono (“l'offset”) è il lavoro pratico della guida-lab; capire *perché* quella struttura funziona è ciò che ti serve sapere all'esame.

Cosa succede dopo dipende dall'indirizzo che ci metti:

- se punta a una **zona non accessibile** → crash con **segmentation fault** (è il DoS);
- se punta a **codice scelto** → controllo del flusso. Da qui partono le due grandi strade: **iniettare** codice nuovo (*shellcode*), oppure **riusare** codice già presente (*ret2libc*, *ROP*).

*(Nota architetturale dal PDF: l'attacco funziona anche su architetture dove lo stack cresce “al contrario” e il buffer si riempie verso gli indirizzi bassi. In quel caso un **strcpy(dest, src)** con **src** troppo lungo sovrascrive l'indirizzo di ritorno della **strcpy** stessa, non quello della funzione che la contiene — ma il principio “l'overflow raggiunge un indirizzo di ritorno” resta identico.)*

I quattro gradini del lab (cosa illustra ciascun binario)

Il lab è costruito come una scala di quattro esercizi a difficoltà crescente. Non li svolgiamo qui (è materia di /lab S4): ti spiego **cosa dimostra ciascuno**, perché è la sequenza logica che lega tutta la teoria.

1. write_var — sovrascrivere una variabile. Il programma ha una variabile locale `control` e un `buf`; per stampare la flag, `control` deve valere `0x42434445`. Sovrascrivendo il buffer si arriva a **riscrivere il valore di control** (il gradino più semplice: scrivo su un dato vicino, senza ancora toccare il flusso). Qui impari due cose che torneranno sempre: che il valore va scritto rispettando il **little-endian** (la `control` giusta si ottiene mettendo `"EDCB"`, cioè `45 44 43 42` al contrario), e che la posizione esatta si trova **per tentativi** (anche con un mini-fuzzing: `for i in {100..150}; do ./es $(perl -e "print 'A'x$i"); done`).

2. secret_function — saltare a una funzione esistente “nascosta”. C'è una funzione `secret()` che stampa la flag (o apre una shell) ma **non viene mai chiamata** dal programma. L'attacco sovrascrive l'indirizzo di ritorno con l'**indirizzo di secret**, trovato con `gdb` (info

functions). È il primo, vero dirottamento di EIP verso codice già presente in `.text`. La variante `secret_function_remote` mostra lo stesso attacco quando il programma è **esposto in rete** (con `nc -l -p 8000 -e ./es`): l'input arriva via socket invece che da `argv` — è il ponte verso uno scenario realistico.

3. shellcode — iniettare codice proprio. Qui non c'è nessuna funzione comoda da richiamare: il codice da eseguire **lo porti tu**, dentro la stringa, sotto forma di *shellcode* (vedi sotto). L'indirizzo di ritorno deve puntare **all'inizio del codice iniettato sullo stack**. È l'attacco “classico” completo, e nel lab lo si abbina al bit SUID-root per ottenere una **shell di root**.

4. returnlib — riusare la libreria C (ret2libc). Quando lo stack è reso non-eseguibile, non puoi più eseguire lo shellcode che inietti. La soluzione è non iniettare codice ma **saltare a una funzione che è già caricata**, in particolare `system("/bin/sh")` della libc. È la risposta diretta alla contromisura NX (vedi sotto).

La visione: questi quattro binari *sono* la mappa concettuale del modulo. Gradino 1: scrivo su un dato. Gradino 2: salto a codice che già esiste. Gradino 3: porto codice nuovo. Gradino 4: quando il gradino 3 è bloccato dalle difese, torno a riusare codice esistente — ma in modo più sofisticato. Tutta la seconda metà della lezione (contromisure e loro bypass) è la spiegazione di *perché* si passa dal gradino 3 al gradino 4.

Lo shellcode (gradino 3, in dettaglio)

Lo **shellcode** è un frammento di codice in linguaggio macchina che l'attaccante inietta come parte della stringa. Il nome viene dal suo scopo tipico: **aprire una shell** (`/bin/sh`) con i privilegi del processo bucato — se il processo è SUID-root, ottieni una **shell di root**.

In piccolo (il meccanismo): un esempio minimo è il codice che invoca la `system` call `exit(0)`. In assembly: `mov EBX, 0` (il parametro), `mov EAX, 1` (1 = numero della syscall `exit`), `int 0x80` (l'interruzione software che su Linux invoca il gestore delle system call). I parametri di una syscall vanno nei registri in ordine `EBX`, `ECX`, `EDX`, `ESI`, `EDI`, e `EAX` contiene il numero della syscall.

Scrivere shellcode iniettabile ha **due trappole** che spiegano dettagli che altrimenti sembrerebbero magia:

- **Zeros cut-off.** Lo shellcode viaggia dentro una stringa, e le funzioni di lettura stringhe interpretano il **byte 0x00 come terminatore**: se il codice macchina contiene un byte 00, la stringa viene troncata lì. Per questo lo shellcode va riscritto con istruzioni equivalenti che **non producano byte nulli** (es. `mov EBX, 0` → `xor EBX, EBX`, che azzerà il registro senza scrivere uno 0 esplicito nel codice).
- **NOP sled (slitta di NOP).** Per far funzionare l'attacco, l'indirizzo di ritorno deve puntare *all'inizio* dello shellcode sullo stack — ma l'indirizzo esatto è difficile da indovinare, perché **lo stack si evolve diversamente a ogni esecuzione** (e dentro `gdb` è ancora diverso che fuori). Soluzione: si fa precedere lo shellcode da una lunga sequenza di istruzioni **NOP** (byte `\x90`, “non fare nulla”). Se l'indirizzo di ritorno cade **in un punto qualsiasi della slitta**, la CPU scorre i NOP senza fare nulla e “scivola” fino allo shellcode. Si allarga così il bersaglio: non devi più colpire un punto, ma un intervallo.

La stringa finale del gradino 3 ha perciò la struttura: **[NOP sled] + [shellcode] + [indirizzo**

di ritorno] (nel lab: 66 NOP + 46 byte di shellcode + 4 byte di indirizzo = 116 byte totali, dove i primi 112 corrispondono a $112*A + 4*B$ trovati cercando l'offset di crash). Lo shellcode “pronto” lo fornisce l'esercitazione; in generale si genera con tool come `msfvenom`.

Gli strumenti (e cosa rivelano)

Gli strumenti non sono accessori: ognuno illumina un pezzo di teoria.

- **gdb (il debugger).** È lo strumento del reverse engineering: un programma per ispezionare un altro programma. I gesti che userai: `disas main / disas vuln` (disassembla una funzione → vedi le istruzioni macchina e dove sta il buffer), `run VALORE` (esegue con un input), `info functions` (elenca le funzioni e i loro indirizzi → così trovi `secret`), `p system` (stampa l'indirizzo di una funzione di libreria, serve per `ret2libc`), `x/200xw $esp` (esamina la memoria a partire dalla cima dello stack → vedi le tue $A = 0x41414141$ e le $B = 0x42424242$ mentre sovrascrivono l'indirizzo di ritorno). Quel $0x41414141$ in un crash è la firma inconfondibile di un overflow riuscito: sono le tue A finite dentro EIP.
- **perl / python per comporre l'input.** Servono a generare byte arbitrari, anche non stampabili: `perl -e 'print "A"x112,"\xNN..."'`. È così che si scrive un indirizzo nella stringa (ricordando il little-endian).
- **I flag di gcc, che sono le contromisure rese tangibili.** Nel lab i binari si compilano disattivando di proposito le difese, e ogni flag corrisponde a una protezione di cui parleremo:
 - `-fno-stack-protector` → **disattiva i canary**;
 - `-z execstack` → **rende lo stack eseguibile** (disattiva NX), necessario per il gradino 3;
 - `-m32` → compila a 32 bit (architettura IA32 del corso; richiede `gcc-multilib`);
 - e separatamente `echo 0 > /proc/sys/kernel/randomize_va_space` → **disattiva l'ASLR**.

La visione, da tenere a mente per l'esame: il fatto che servano *tutti questi flag* per far funzionare l'attacco classico ti dice già la conclusione del modulo: **oggi un buffer overflow “alla vecchia maniera” è raro**, perché di default canary + NX + ASLR sono accesi. Gli attacchi moderni (`ret2libc`, ROP) nascono proprio per aggirare quelle difese.

Le contromisure (e come si aggirano)

Questa è la sezione più “da quiz”. Per ognuna: cosa fa, chi la fornisce (compilatore? OS? hardware?), e il suo punto debole.

Canary (“canarino”)

Cosa fa. Prima di un buffer, sullo stack viene messo un **valore di riferimento casuale** (il “canarino”, come quelli dei minatori che rilevavano le fughe di gas). Al ritorno dalla funzione, il programma **verifica che il canarino sia integro**: se un overflow l'ha sovrascritto, la verifica fallisce e il processo viene **terminato** (di default; l'evento può anche essere catturato via segnali dell'OS). L'attaccante non può “scavalcare” il canarino: per arrivare all'indirizzo di ritorno deve passarci sopra, e così lo altera.

Chi la fornisce. Compilatore + libreria standard (in GCC: patch *ProPolice*, poi nel ramo principale dalla 4.1). **Non** è una protezione dell'OS o dell'hardware. Si attiva con `-fstack-protector` (solo buffer di stringhe) o `-fstack-protector-all` (tutti); `--param ssp-buffer-size=` fissa una soglia oltre cui proteggere, per limitare l'**overhead**.

Punto debole (quiz!). È aggirabile a **forza brutta** in programmi che fanno *fork* di figli: il figlio eredita **lo stesso canarino** del padre; provo a indovinarlo un pezzo alla volta — se sbaglio il figlio crasha, ne genero un altro (canarino identico) e riprovo. Generalizzazione difensiva: **CFI** (vedi sotto).

NX / W^X (stack non eseguibile)

Cosa fa. Marca le pagine di memoria dello stack come **non eseguibili**: anche se l'attaccante inietta shellcode sullo stack, la CPU **rifiuta di eseguirlo**. È la difesa diretta contro il gradino 3 (shellcode).

Chi la fornisce. È una **feature hardware** (Intel la chiama **XD bit**, *eXecute Disable*), che però **dev'essere supportata dall'OS**. Attenzione a un punto che il PDF sottolinea: **non esiste sui processori Intel/AMD a 32 bit puri** — è arrivata solo coi processori a 64 bit; un sistema x86 a 32 bit può sfruttarla solo se gira su un processore x86_64. Linux la supporta dal kernel 2.6.8 (patch PaX/grsecurity). **W^X** (“Write XOR eXecute”) è l'evoluzione: ogni pagina è *o* scrivibile *o* eseguibile, **mai entrambe** — più rigida del solo XD bit (in Linux implementata in parte da ExecShield).

Punto debole (quiz!). NX/W^X sono **quasi inutili da soli**, perché diversi attacchi fanno un uso *legittimo* dello stack: non eseguono codice *dallo* stack, ma lo usano per **riusare codice già marcato eseguibile** altrove. È esattamente ciò che fanno ret2libc, ret2syscall e ROP.

ASLR (Address Space Layout Randomization)

Cosa fa. Rende casuale l'**indirizzo di partenza dei segmenti** del processo a ogni esecuzione, così l'attaccante non sa più *dove* puntare. Senza un indirizzo prevedibile, non può scrivere un indirizzo di ritorno valido né sapere dove sia lo shellcode.

Chi la fornisce. L'OS (Linux: patch grsecurity/PaX, inclusa nel kernel solo in alcune distro). Randomizza gli **indirizzi virtuali**, non la disposizione fisica in RAM.

Punto debole (quiz! — è il dettaglio che il PDF ripete). L'ASLR è **incompleto**: randomizza librerie, base dello stack e base dello heap, **ma NON il segmento .text**. Quindi il codice del programma resta a indirizzi **prevedibili** — ed è lì che gli attacchi moderni vanno a pescare i pezzi di codice da riusare. È inoltre attaccabile a forza brutta (difficile sul 64 bit, dove lo spazio è enorme) o aggirabile usando i puntatori del codice legittimo. **Estensione difensiva: PIE.**

PIE e RelRO (rafforzamenti dell'ASLR)

- **PIE** (*Position Independent Executable*): rende randomizzato **anche .text**, chiudendo il buco dell'ASLR. Per funzionare richiede una **doppia indirezione** dei puntatori a funzione tramite le tabelle **PLT** e **GOT** — che però sono a loro volta **sovrascrivibili** per dirottare le chiamate.
- **RelRO** (*Relocation Read-Only*): hardening che rende quelle tabelle (in particolare la GOT) **in sola lettura** dopo il caricamento, per chiudere quel nuovo buco.

CFI (Control Flow Integrity)

Cosa fa. È la **generalizzazione** di tutte le difese precedenti: una *famiglia* di tecniche (almeno 14 implementazioni note) che mira a **garantire che EIP non possa mai essere controllato da un attaccante**. Un esempio rilevante è il **supporto hardware per puntatori cifrati**: il processo sceglie all'avvio una chiave e la CPU **cifra/decifra trasparentemente** tutti gli indirizzi di ingresso/ritorno dalle funzioni; un programma vulnerabile può essere mandato in crash (DoS) ma **non dirottato**. Esempio reale citato: i **PAC** (*Pointer Authentication Codes*) sui processori Apple >= A12.

Gli attacchi “anti-difesa”: riusare il codice esistente

Quando NX impedisce di eseguire shellcode sullo stack, l'idea cambia: **non iniettare codice, ma riusare codice già caricato** (in `libc` o in `.text`, che ricordiamo l'ASLR *non* randomizza). Lo stack overflow serve ancora, ma ora per iniettare **solo dati** — indirizzi e parametri — non istruzioni.

- **ret2libc.** Si dirotta EIP su una **funzione di libreria già caricata**, tipicamente `system("/bin/sh")`. Servono tre cose, che si trovano col debugger: l'**indirizzo di system** (p `system` in gdb), un modo di mettere sullo stack la **stringa /bin/sh** (spesso è già nell'ambiente, nella variabile `SHELL`), e comporre lo stack così che, al RET, `system` trovi al posto giusto il suo parametro. Si possono concatenare più ritorni (es. far seguire `exit` per una chiusura pulita che non insospettisca). *Due dettagli pratici dal lab:* va aggiunto `SHELL=` (6 caratteri) per puntare al valore giusto della variabile, e se l'indirizzo di `system` finisce con un byte `00` lo si “ritocca” col byte successivo (`04/08`), perché lo `00` troncherebbe la stringa. Nota importante: **ret2libc funziona anche con lo stack non eseguibile** — è la sua ragion d'essere.
- **ret2syscall.** Variante: invece di saltare a una funzione di libreria, si **innesca direttamente una system call** (caricare il numero in `EAX`, i parametri nei registri, e invocare `INT 0x80`). Poiché queste operazioni sono comuni, in `.text` (non randomizzato!) esistono di sicuro frammenti di codice come `POP EAX; RET` o `INT 0x80` da riusare.
- **ROP (Return-Oriented Programming).** La generalizzazione massima. Si setaccia `.text` cercando **gadget**: brevi sequenze di istruzioni che **terminano con un RET** (es. `pop edi; ret`). Sullo stack si impila una **catena di indirizzi di gadget** intervallati dai dati: ogni gadget esegue il suo pezzettino, consuma i suoi dati e col RET “salta al gadget successivo” della catena. In pratica si **assembla un intero programma** mettendo in fila pezzi di codice già presenti — senza iniettare nulla e senza eseguire dallo stack, quindi **immune a NX**. Si può perfino sfruttare il **disallineamento**: leggendo i byte di `.text` con un offset diverso, si ottengono istruzioni del tutto nuove rispetto a quelle “intese” dal compilatore.

Esistono anche **Format Strings** (sfruttare una stringa di formato passata a `printf` per leggere/scrivere memoria) e **Heap Overflow** (sfruttare i metadati che la `libc` usa per l'allocazione dinamica), citati come ulteriori famiglie ma non centrali nel lab.

Connessioni

- **Con SysAdmin (2A — utenti e permessi):** la potenza di questi attacchi dipende dai **privilegi del processo bucato**. Un binario con bit **SUID-root** (`chmod u+s`) che ha un

buffer overflow regala una **shell di root**: è la stessa misconfigurazione di permessi che studi in privilege escalation. Il gradino 3 del lab attiva proprio il SUID per mostrarlo.

- **Con S1 (enumerazione)**: la variante `secret_function_remote` espone il binario in rete (`nc`); prima di attaccarlo lo enumeri come in S1 — e gli IP del PDF (192.168.56.x) sono **esempi**, da verificare sempre con `ip a`.
 - **Con S5 (firewall, prossimo modulo)**: se un servizio vulnerabile è esposto, un firewall che ne blocca la porta è una mitigazione *di rete* complementare alle difese *di memoria* viste qui.
 - **Con S11 (privilege escalation, tipo d'esame “integrity/privesc”)**: un binary exploit è uno dei modi per scalare i privilegi; concettualmente vicino alla tipologia capstone.
-

Domande di autoverifica (stile quiz teorico)

Ricorda: all'esame c'è **penalità** per risposta sbagliata. Se non sei sicuro, **non rispondere**. Prova a giustificare ogni risposta, non solo a indovinarla.

1. **Vero o Falso**: lo stack cresce verso indirizzi più alti, mentre un buffer al suo interno si riempie verso indirizzi più bassi.
2. **Scelta multipla — la protezione NX (stack non eseguibile) è inefficace da sola perché**:
 - (a) i canary la disattivano; (b) molti attacchi (ret2libc, ROP) **riusano** codice già eseguibile invece di eseguire dallo stack; (c) randomizza solo `.text`; (d) introduce troppo overhead.
3. **Vero o Falso**: l'ASLR randomizza l'indirizzo di tutti i segmenti del processo, incluso `.text`.
4. **Domanda aperta**: spiega perché un indirizzo come 0x0804D432 viene inserito nella stringa d'attacco come `\x32\xD4\x04\x08`.
5. **Scelta multipla — i canary sono forniti da**: (a) l'hardware; (b) l'OS; (c) il compilatore + la libreria standard; (d) il protocollo di rete.
6. **Domanda aperta**: a cosa serve un NOP sled in un attacco con shellcode, e quale problema concreto risolve?
7. **Vero o Falso**: ret2libc richiede che lo stack sia eseguibile per funzionare.

*(Risposte sintetiche per autocorrezione: 1-F: è il contrario — lo stack cresce verso il basso, il buffer si riempie verso l'alto; 2-b; 3-F: l'ASLR **non** randomizza `.text`, ed è proprio il buco che PIE chiude; 4: little-endian, Intel memorizza dal byte meno significativo; 5-c; 6: allarga il bersaglio dell'indirizzo di ritorno — non dovendo colpire l'esatto inizio dello shellcode, basta “atterrare” in un punto qualsiasi della slitta di NOP e scivolare fino al codice, risolvendo l'imprevedibilità dell'indirizzo dello stack; 7-F: al contrario, ret2libc nasce proprio per funzionare **quando lo stack NON è eseguibile**.)*

Collegati

- [\[\[appunti_moduloS4_binary_exploits\]\]](#)
- [\[\[guida_lab_moduloS4_binary_exploits\]\]](#)

Hub: [\[\[master_map_studio\]\]](#) · [\[\[concept_maps\]\]](#) · [\[\[metodo_studio_esami_pratici\]\]](#)